

1	2	1	2	4	3	6
---	---	---	---	---	---	---



问题：查找「目标峰顶」位置

虽然存在多个峰顶，
但我们可以假设要找的就是其中一个

1	2	1	2	4	3	6
↑		↑			↑	
<i>L</i>		<i>M</i>			<i>R</i>	

红色背景表示 false，即目标峰顶左侧
蓝色背景表示 true，即目标峰顶及其右侧
 白色背景表示不确定

根据这一定义，6 必然是蓝色

然后通过比较 M 和 $M + 1$ 指向的元素大小，
来判断目标峰顶的位置

1	2	1	2	4	3	6
			↑	↑	↑	
			<i>L</i>	<i>M</i>	<i>R</i>	

峰值元素是指其值严格大于左右相邻值的元素。

给你一个整数数组 `nums`，找到峰值元素并返回其索引。数组可能包含多个峰值，在这种情况下，返回 任何一个峰值 所在位置即可。

你可以假设 `nums[-1] = nums[n] = -∞`。

你必须实现时间复杂度为 $O(\log n)$ 的算法来解决此问题。

示例 1:

输入: `nums = [1,2,3,1]`

输出: 2

解释: 3 是峰值元素，你的函数应该返回其索引 2。

示例 2:

输入: `nums = [1,2,1,3,5,6,4]`

输出: 1 或 5

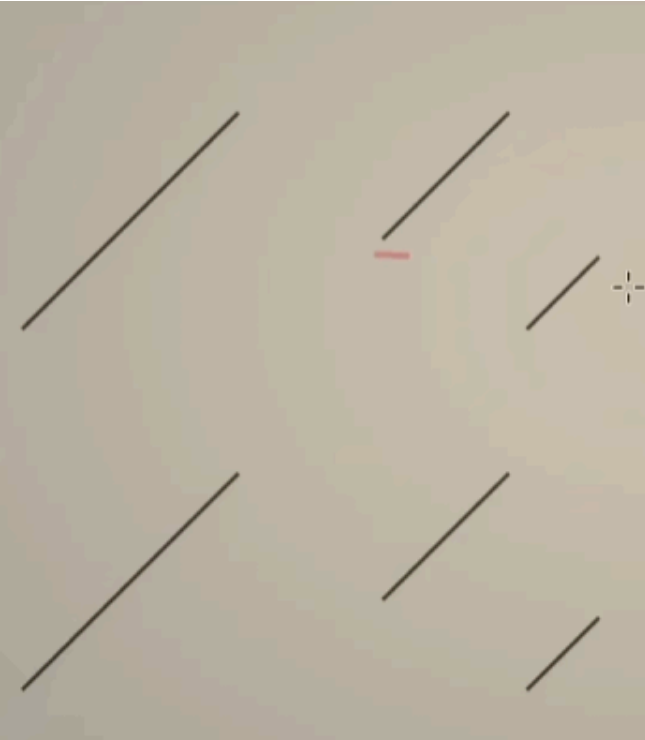
解释: 你的函数可以返回索引 1，其峰值元素为 2；
或者返回索引 5，其峰值元素为 6。

提示:

- `1 <= nums.length <= 1000`
- `-231 <= nums[i] <= 231 - 1`
- 对于所有有效的 `i` 都有 `nums[i] != nums[i + 1]`

```
class Solution(object):
    def findPeakElement(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        left = -1
        right = len(nums) - 1
        while left + 1 < right:
            mid = (left + right) // 2
            if nums[mid] > nums[mid+1]:
                right = mid
            else:
```

```
        left = mid  
    return right
```



红色背景表示 false, 即最小值左侧
蓝色背景表示 true, 即最小值及其右侧

根据这一定义, $n - 1$ 必然是蓝色

153. 寻找旋转排序数组中的最小值

中等

🏷 相关标签

🏢 相关企业

💡 提示

Aa

已知一个长度为 n 的数组，预先按照升序排列，经由 1 到 n 次旋转后，得到输入数组。例如，原数组 `nums =`

`[0,1,2,4,5,6,7]` 在变化后可能得到：

- 若旋转 4 次，则可以得到 `[4,5,6,7,0,1,2]`
- 若旋转 7 次，则可以得到 `[0,1,2,4,5,6,7]`

注意，数组 `[a[0], a[1], a[2], ..., a[n-1]]` 旋转一次的结果为数组 `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`。

给你一个元素值互不相同的数组 `nums`，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的最小元素。

你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1：

输入：`nums = [3,4,5,1,2]`

输出：`1`

解释：原数组为 `[1,2,3,4,5]`，旋转 3 次得到输入数组。

示例 2：

输入：`nums = [4,5,6,7,0,1,2]`

输出：`0`

解释：原数组为 `[0,1,2,4,5,6,7]`，旋转 4 次得到输入数组。

示例 3：

输入：`nums = [11,13,15,17]`

输出：`11`

解释：原数组为 `[11,13,15,17]`，旋转 4 次得到输入数组。

提示：

- `n == nums.length`
- `1 <= n <= 5000`

• `5000 <= nums[i] <= 5000`

- `-5000 <= nums[1] <= 5000`
- `nums` 中的所有整数 互不相同
- `nums` 原来是一个升序排序的数组，并进行了 `1` 至 `n` 次旋转

```
class Solution(object):
    def findMin(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        left = -1
        right = len(nums) - 1
        while left + 1 < right:
            mid = (left + right) // 2
            if nums[mid] < nums[-1]:
                right = mid
            else:
                left = mid
        return nums[right]
```

33. 搜索旋转排序数组

已解答

中等

🏷 相关标签

🏢 相关企业

⌘

整数数组 `nums` 按升序排列，数组中的值 互不相同。

在传递给函数之前，`nums` 在预先未知的某个下标 `k` ($0 \leq k < \text{nums.length}$) 上进行了 向左旋转，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (下标从 0 开始 计数)。例如，`[0,1,2,4,5,6,7]` 下标 3 上向左旋转后可能变为 `[4,5,6,7,0,1,2]`。

给你 旋转后 的数组 `nums` 和一个整数 `target`，如果 `nums` 中存在这个目标值 `target`，则返回它的下标，否则返回 `-1`。

你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1:

输入: `nums = [4,5,6,7,0,1,2]`, `target = 0`
输出: 4

示例 2:

输入: `nums = [4,5,6,7,0,1,2]`, `target = 3`
输出: -1

示例 3:

输入: `nums = [1]`, `target = 0`
输出: -1

提示:

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` 中的每个值都 独一无二
- 题目数据保证 `nums` 在预先未知的某个下标上进行了旋转
- $-10^4 \leq \text{target} \leq 10^4$

```
class Solution(object):
    def search(self, nums, target):
        """
        :type nums: List[int]
        :type target: int
        :rtype: int
```

```
"""
def is_blue(i):
    end =nums[-1]
    if nums[i] \> end:
        return target \> end and nums[i] \>=target
    else:
        return target \> end or nums[i] \>=target
left = -1
right = len(nums)
while left +1 \< right:
    mid = (left + right) //2
    if is_blue(mid):
        right = mid
    else:
        left = mid
if right == len(nums) or nums[right] != target:
    return -1
return right
```